

UNIVERSITÉ HASSAN II DE CASABLANCA
École Nationale Supérieure d'Arts et Métiers
Département Informatique & Intelligence Artificielle

RAPPORT DE PROJET MÉTIER

Orbyte

Gestion de projets collaborative augmentée par l'IA

Filière :
Génie Informatique & Intelligence Artificielle (IAGI)

Session :
Année Universitaire 2025 – 2026

Réalisé par :

Encadré par :

HAFID Salma

Pr. JADLI Aissam

HANI Hiba

MAJOUG Ilyass

QUAISSE Marouane

Casablanca, Maroc

Remerciements

Nous tenons à exprimer notre profonde gratitude à notre encadrant, **Monsieur le Professeur JADLI Aissam**, pour sa disponibilité, la pertinence de ses conseils et la rigueur de son suivi tout au long de ce projet. Ses orientations techniques et méthodologiques ont été essentielles pour surmonter les défis de ce travail et ont largement contribué à notre progression académique et professionnelle.

Nous remercions également l'ensemble du corps professoral de l'ENSAM Casablanca pour la qualité de la formation dispensée, qui nous a fourni les bases nécessaires à la réalisation de ce projet métier.

Les auteurs

**HANI Hiba, HAFID Salma,
MAJOUG Ilyass, QUAISSE Marouane**

Résumé

Ce rapport documente la conception et l'implémentation de **Orbyte**, une plateforme de gestion de projets logiciels conçue pour allier méthodologies agiles et intelligence artificielle. Développé dans le cadre du module Projet Métier à l'ENSAM Casablanca, ce système répond au besoin croissant des petites équipes techniques pour un outil à la fois léger et doté de fonctionnalités d'automatisation avancées.

L'innovation majeure réside dans l'intégration d'un assistant IA capable d'interagir avec le cycle de vie des tickets (Scrum/Kanban) en langage naturel, facilitant ainsi la génération de contenu et l'analyse prédictive de la santé des projets. Sur le plan architectural, la solution repose sur une approche microservices performante utilisant **Spring Boot** pour le backend, **React** et **TypeScript** pour le frontend, ainsi qu'un service IA spécialisé sous **FastAPI**.

L'ensemble de l'infrastructure est déployé de manière automatisée sur **Hetzner Cloud** via une pipeline CI/CD complète, orchestrée par **Terraform** et **Ansible**. Ce travail illustre une démarche d'apprentissage approfondie sur l'industrialisation des logiciels modernes, l'exploitation des LLMs en milieu professionnel.

Mots-clés : Gestion de projets, Intelligence Artificielle, Microservices, Spring Boot, React, DevOps, CI/CD.

Abstract

This report details the design and implementation of **Orbyte**, an AI-augmented project management platform tailored to bridge the gap between agile methodologies and artificial intelligence. Developed as part of the "Projet Métier" module at ENSAM Casablanca, this system addresses the growing need for small technical teams to have a lightweight yet powerful tool equipped with advanced automation features.

The core innovation lies in the integration of an AI assistant capable of interacting with the ticket lifecycle (Scrum/Kanban) through natural language processing, thus facilitating content generation and predictive project health analysis. Architecturally, the solution follows a high-performance microservices approach, utilizing **Spring Boot** for the backend, **React and TypeScript** for the frontend, and a specialized **FastAPI** service for AI functionalities.

The entire infrastructure is automatically deployed on **Hetzner Cloud** through a comprehensive CI/CD pipeline, orchestrated by **Terraform** and **Ansible**. This project highlights a significant learning curve in modern software industrialization, and the practical integration of Large Language Models (LLMs) in a professional environment.

Keywords : Project Management, Artificial Intelligence, Microservices, Spring Boot, React, DevOps, CI/CD.

Table des matières

Remerciements	1
Résumé	2
Abstract	3
Liste des figures	8
Liste des tableaux	9
1 Introduction générale	11
1.1 Contexte général	11
1.2 Problématique	11
1.3 Objectifs du projet	11
1.4 Organisation du rapport	12
2 Analyse des besoins et spécifications	13
2.1 Identification des acteurs	13
2.2 Diagramme de cas d'utilisation	13
2.3 Spécifications fonctionnelles — Méthode MoSCoW	14
2.4 Spécifications non fonctionnelles	17
3 Conception et architecture	19
3.1 Vue globale de l'architecture	19
3.2 Schéma d'architecture microservices	19
3.3 Modèle de données	20
3.3.1 Hiérarchie des entités	20
3.3.2 Tableau des entités principales	20

3.3.3	Énumérations métier	22
3.4	Patterns architecturaux adoptés	22
4	Implémentation Backend — Spring Boot	23
4.1	Stack technique	23
4.2	Structure du projet	24
4.3	Sécurité JWT et MFA	25
4.3.1	Filtre d'authentification JWT	25
4.3.2	Configuration Spring Security	27
4.4	Endpoints API principaux	28
5	Implémentation Frontend — React	31
5.1	Stack technique frontend	31
5.2	Architecture des pages et routage	32
5.3	Composants clés	33
5.4	Implémentation du Kanban — dnd-kit	34
5.5	Gestion des appels API et authentification frontend	34
6	Intelligence Artificielle	36
6.1	Vision fonctionnelle — Ce que l'IA apporte à l'utilisateur	36
6.1.1	Mission 1 — Comprendre instantanément n'importe quel projet de code	36
6.1.2	Mission 2 — Créer des éléments de projet structurés par le texte (Wizard multi-entités)	37
6.1.3	Mission 3 — Générer du contenu ancré dans le contexte réel du code	38
6.2	Les fonctionnalités IA — Vue d'ensemble	38
6.3	Architecture du service IA	40
6.3.1	Sécurité des Personal Access Tokens GitHub	41
6.4	Les endpoints REST exposés	42
6.5	Intégration GitHub — github_service.py	43
6.6	Pipeline de génération contextuelle	44
6.7	Génération d'entités par intention	45
6.7.1	Mode dynamique — Swagger-driven	46
6.7.2	Mode fallback statique	46
6.7.3	Gestion de la hiérarchie et résolution de dépendances	46

6.7.4	Génération de contenu agile — Ask AI	47
7	DevOps & Déploiement	48
7.1	Vue d'ensemble de l'infrastructure	48
7.2	Architecture générale	48
7.2.1	Topologie réseau	48
7.2.2	Structure du dépôt d'infrastructure	50
7.3	Stratégie de conteneurisation	50
7.4	Infrastructure as Code — Terraform & Ansible	51
7.4.1	Phase 0 — Préparation locale	51
7.4.2	Phase 1 — Provisionnement Cloud avec Terraform	51
7.4.3	Phase 2 — Configuration des serveurs avec Ansible	52
7.4.4	Phase 3 — Déploiement applicatif avec Docker Compose	52
7.5	Sécurité de l'infrastructure	53
7.5.1	Isolation réseau	53
7.5.2	Gestion des secrets	53
7.5.3	Configuration HTTPS/SSL et domaine	53
7.6	Pipeline CI/CD — GitHub Actions	54
7.7	Procédures opérationnelles	54
7.7.1	Déploiement initial	54
7.7.2	Mise à jour applicative	55
7.7.3	Destruction de l'infrastructure	55
7.8	Évolutions prévues	55
8	Tests et validation	56
8.1	Stratégie globale de tests	56
8.2	Tests backend	56
8.2.1	Tests d'intégration via requêtes HTTP	56
8.2.2	Validation de la logique de sécurité	57
8.3	Tests frontend	57
8.3.1	Vérification statique TypeScript	57
8.3.2	Analyse statique ESLint	57
8.3.3	Tests fonctionnels manuels	58
8.4	Tableau récapitulatif des tests	58
	Conclusion générale	60

A	Diagramme de classes UML	62
B	Entités JPA — Champs complets	64
C	Cahier des charges (extrait)	66

Table des figures

2.1	Diagramme de cas d'utilisation simplifié — Orbyte	14
3.1	Architecture microservices d'Orbyte	19
6.1	Pipeline de génération contextuelle à double LLM (rag_service.py) . .	45
7.1	Architecture de la topologie réseau — Hetzner Cloud	49
7.2	Pipeline de déploiement en cinq phases — PM-B-infra	51
7.3	Pipeline CI/CD — GitHub Actions vers Hetzner Cloud	54
A.1	Diagramme de classes UML complet — Orbyte	63

Liste des tableaux

3.2	Énumérations JPA du modèle de données	22
4.1	Dépendances principales du backend	24
5.1	Stack technique frontend (extrait <code>package.json</code>)	32
5.2	Pages de l'application et leur statut d'accès	33
6.2	Fonctionnalités IA et intégration utilisateur	40
6.3	Dépendances du service IA (extrait <code>requirements.txt</code>)	41
6.4	Endpoints REST du service IA (<code>main.py</code>)	43
6.5	Intentions de création et endpoints correspondants	47
7.1	Topologie des nœuds Hetzner Cloud — PM-B-infra	49
7.2	Règles de pare-feu par composant	52
7.3	Évolutions prévues de l'infrastructure	55
8.1	Fichiers de tests HTTP backend	57

Chapitre 1

Introduction générale

1.1 Contexte général

La gestion de projets logiciels pour les petites et moyennes équipes (de 2 à 50 personnes) représente un défi quotidien. Les outils existants tels que JIRA ou ClickUp sont fréquemment perçus comme trop complexes, trop coûteux ou trop génériques pour répondre aux besoins spécifiques des équipes de développement de taille modeste. Par ailleurs, l'essor fulgurant des modèles de langage (LLMs) ouvre une opportunité inédite, à savoir intégrer une couche d'intelligence artificielle directement au cœur du flux de travail des équipes techniques.

C'est dans ce contexte que s'inscrit le projet **Orbyte**.

1.2 Problématique

L'analyse de ce contexte conduit à poser la problématique centrale suivante :

Comment concevoir une plateforme de gestion de projets légère, sécurisée et intelligente qui améliore la productivité des équipes techniques, tout en restant simple à déployer et à maintenir ?

1.3 Objectifs du projet

Ce projet vise à concevoir, développer et déployer une plateforme de gestion de projets collaborative, moderne et augmentée par l'IA. Les objectifs principaux sont les suivants :

- Fournir une interface intuitive pour la création et le suivi des tickets et projets (Kanban, Scrum, sprints, releases).
- Intégrer un assistant IA conversationnel capable d’agir en langage naturel au sein du workflow.
- Assurer une architecture robuste, sécurisée et déployable en production via une pipeline CI/CD complète.
- Proposer des fonctionnalités avancées d’analyse de santé de projet et de génération automatique de contenu.

1.4 Organisation du rapport

Ce rapport est structuré en plusieurs chapitres, chacun pris en charge par un membre de l’équipe selon sa responsabilité technique :

Chapitre 2 — Analyse des besoins

Spécifications fonctionnelles et non fonctionnelles, cas d’utilisation.

Chapitre 3 — Conception & Architecture

Diagrammes UML, modèle de données, architecture technique.

Chapitre 4 — Implémentation Backend

Spring Boot, API REST, sécurité JWT/MFA, WebSocket.

Chapitre 5 — Implémentation Frontend

React + TypeScript, boards Kanban/Scrum, interface utilisateur.

Chapitre 6 — Intelligence Artificielle

Assistant conversationnel, génération de tickets, recherche sémantique.

Chapitre 7 — DevOps & Déploiement

CI/CD, Terraform, Ansible, Hetzner Cloud, monitoring.

Chapitre 8 — Tests & Validation

Tests unitaires, d’intégration, E2E (Playwright).

Conclusion & Perspectives

Bilan du projet et pistes d’amélioration.

Chapitre 2

Analyse des besoins et spécifications

2.1 Identification des acteurs

La première étape de l'analyse consiste à identifier les différents utilisateurs qui interagiront avec la plateforme. Orbyte distingue trois acteurs principaux, chacun possédant un périmètre d'action bien délimité.

- **Administrateur / Propriétaire de Workspace** : crée et configure les espaces de travail (**Workspace**), invite des membres via email, définit les rôles (**OWNER**, **ADMIN**, **MEMBER**) et gère l'ensemble des ressources hiérarchiques (**Spaces**, **Folders**, **Lists**, **Sprints**).
- **Chef de projet / Scrum Master** : planifie les **Sprints**, crée et assigne les tâches (**Task**), supervise l'avancement via les vues **Kanban** et **Sprint**, et utilise l'assistant IA pour générer du contenu structuré.
- **Développeur / Membre de l'équipe** : consulte ses tâches assignées, met à jour les statuts par glisser-déposer, interagit avec l'assistant IA pour analyser les dépôts **GitHub** liés au projet.

2.2 Diagramme de cas d'utilisation

Le diagramme suivant représente les principales interactions entre les acteurs et le système Orbyte .

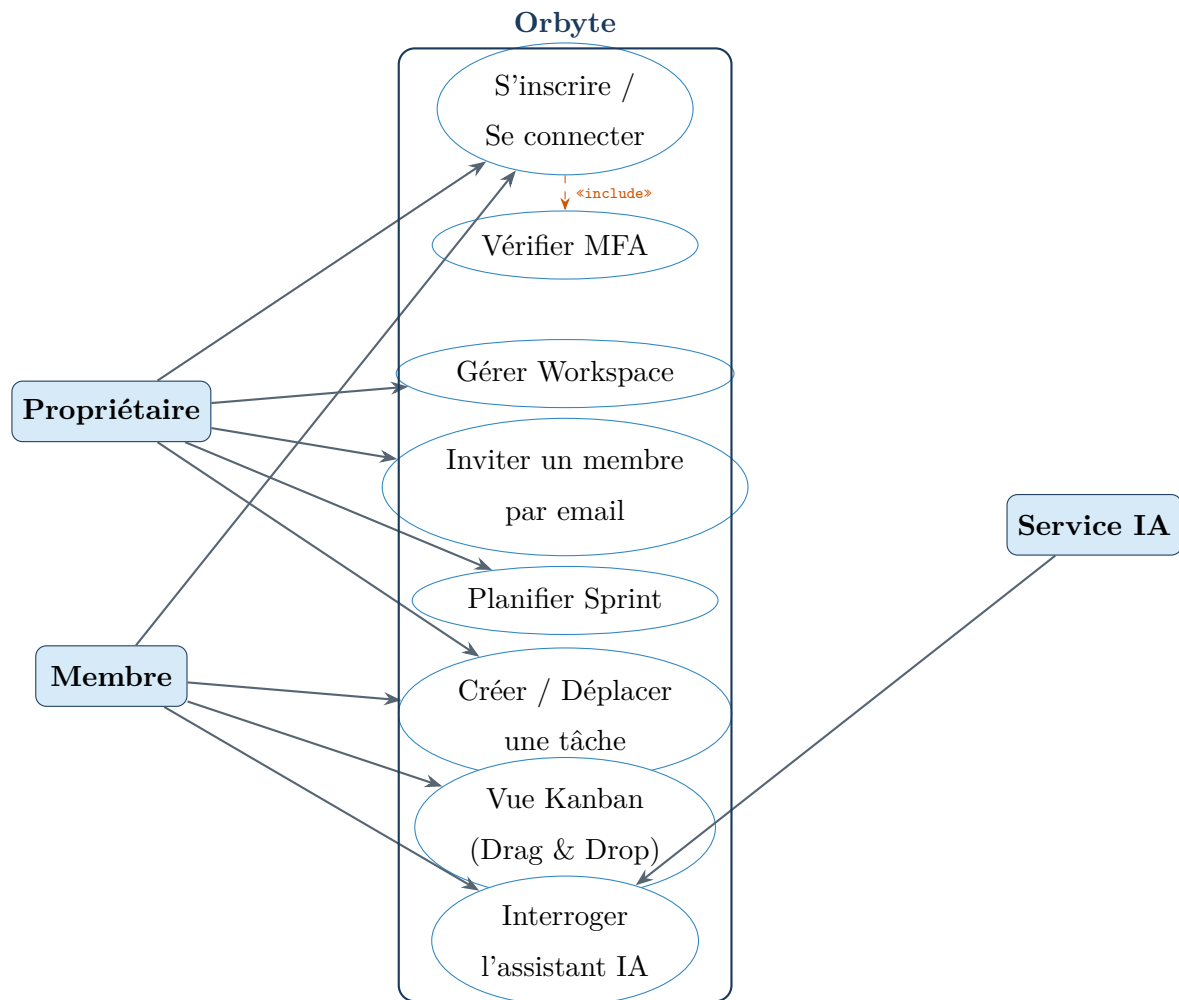


FIGURE 2.1 – Diagramme de cas d'utilisation simplifié — Orbyte

2.3 Spécifications fonctionnelles — Méthode MoS-CoW

Les fonctionnalités de la plateforme ont été priorisées selon la méthode MoSCoW afin de garantir une livraison incrémentale cohérente avec les contraintes du module Projet Métier.

Fonctionnalité	Description	Priorité	Acteur
Authentification JWT	Inscription, connexion avec émission d'un Access Token et Refresh Token signés (jjwt 0.11.5).	Must Have	Tous
MFA par OTP email	Génération d'un code OTP temporaire haché (BCrypt) avec gestion de challenge et tentatives.	Must Have	Tous
Gestion Workspace	Création, modification, suppression d'un Workspace avec slug unique.	Must Have	Propriétaire
Invitation par email	Envoi d'un lien d'invitation sécurisé via POST /api/workspaceMembers/invite.	Must Have	Propriétaire
Gestion des rôles	Attribution de rôles OWNER / ADMIN / MEMBER via WorkspaceRole.	Must Have	Propriétaire
Gestion Space & Folder	Création de Space (avec couleur, visibilité privée) et de Folder imbriqués.	Must Have	Propriétaire
Gestion Liste	Création de listes de type SPRINT ou PHASE avec ordre d'affichage.	Must Have	Chef projet
Gestion Sprint	Création de Sprint avec dates ISO 8601, objectif et statut actif.	Must Have	Chef projet

Suite page suivante

Fonctionnalité	Description	Priorité	Acteur
Gestion Task	CRUD complet sur Task : titre, description, statut {TO_DO, IN_DEV, IN_TEST, IN_REVIEW, DONE}, priorité {LOW, MEDIUM, HIGH, URGENT}, date d'échéance, assignation.	Must Have	Tous
Vue Kanban (dnd-kit)	Déplacement de tâches par Drag & Drop entre colonnes avec mise à jour instantanée du statut via l'API.	Must Have	Tous
Tableau de bord	Page DashboardPage avec panneau de ressources, sidebar, navigation multi-vues.	Must Have	Tous
OAuth GitHub	Flux OAuth 2.0 via GitHubOAuthController permettant de lier un compte GitHub.	Should Have	Tous
Assistant IA conversationnel	Interface AIPage permettant de poser des questions en langage naturel sur un dépôt GitHub.	Should Have	Tous
Génération d'entités par IA	Détection d'intention + génération JSON structuré via POST /api/ia/generate (RAG + OpenRouter).	Should Have	Tous
Conversations persistantes	Sauvegarde des échanges IA via Conversation et ConversationMessage.	Should Have	Tous

Suite page suivante

Fonctionnalité	Description	Priorité	Acteur
Commentaires & Sous-tâches	Entités Comment, Subtask, Attachment liées aux tâches.	Could Have	Membres
Notifications	Entité Notification liée à l'utilisateur pour les alertes applicatives.	Could Have	Tous
Documentation Swagger	Interface OpenAPI générée automatiquement via Springdoc (accessible à /swagger-ui).	Could Have	Développeurs

2.4 Spécifications non fonctionnelles

Au-delà des fonctionnalités métiers, plusieurs exigences non fonctionnelles encadrent la qualité globale du système.

Performance

L'architecture SPA (React + Vite) garantit un chargement initial rapide et des transitions sans rechargement. Le backend adopte une politique STATELESS (aucune session serveur) pour maximiser la scalabilité horizontale.

Sécurité

Les mots de passe sont hachés via BCrypt. Chaque requête est authentifiée par un JWT signé. La configuration CORS n'accepte que les origines autorisées. Le MFA ajoute une couche de protection supplémentaire contre les accès non autorisés.

Maintenabilité

L'architecture en couches (Controller / Service / Repository) et l'usage de Lombok et ModelMapper minimisent le code répétitif et facilitent l'évolution du modèle de données.

Disponibilité

L'ensemble de la plateforme est conteneurisée (Docker) et déployée sur Hetzner Cloud via une pipeline CI/CD GitHub Actions, garantissant des mises en production reproductibles et rapides.

Interopérabilité

L'API REST est documentée via OpenAPI 3 (Springdoc 2.5.0) et consommée aussi bien par le frontend React que par le microservice IA Python.

Chapitre 3

Conception et architecture

3.1 Vue globale de l'architecture

La plateforme Orbyte repose sur une architecture distribuée en trois services indépendants communiquant via des interfaces REST bien définies. Cette séparation garantit l'isolation des responsabilités, la scalabilité individuelle de chaque composant et la facilité de déploiement en conteneurs.

3.2 Schéma d'architecture microservices

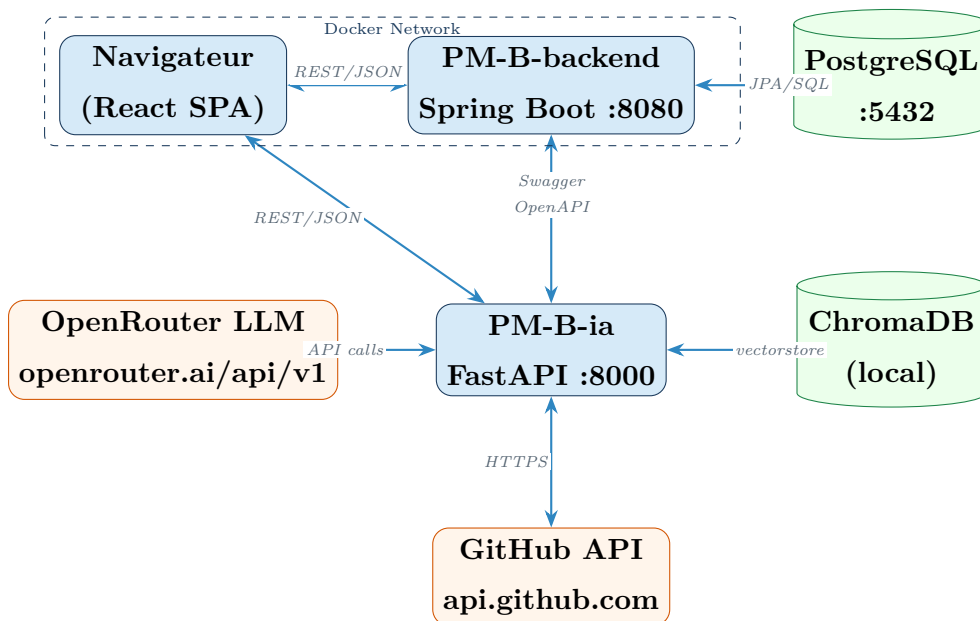


FIGURE 3.1 – Architecture microservices d'Orbyte

Remarque

Le service IA interroge dynamiquement l'endpoint `/v3/api-docs` du backend Spring Boot pour obtenir la documentation OpenAPI à jour, lui permettant de choisir le bon endpoint `POST` lors de la génération d'entités. Cette approche *Swagger-driven generation* évite tout couplage statique entre les deux services.

3.3 Modèle de données

Le modèle relationnel d'Orbyte reflète fidèlement la hiérarchie organisationnelle d'un outil de gestion de projets Agile. Il est composé de **12 entités JPA** persistées dans une base PostgreSQL.

3.3.1 Hiérarchie des entités

Workspace → Space → Folder → Liste → Task

Un `Sprint` peut être associé directement à une `Liste` (via la relation `ManyToOne sprint_id`) et regroupe les tâches à réaliser durant une itération.

3.3.2 Tableau des entités principales

Entité	Champs clés	Relations
User	id (UUID), firstName, lastName, email (unique), password (BCrypt), role, mfaEnabled, mfaOtpCodeHash, mfaOtpExpiresAt, avatarUrl, createdAt	OneToMany → WorkspaceMember, Task (assignee), Comment, Notification, Subtask
Workspace	id (UUID), name, slug, createdAt	ManyToOne → User (owner); OneToMany → WorkspaceMember, Space

Entité	Champs clés	Relations
Workspace-Member	id (UUID), role (WorkspaceRole), joinedAt	ManyToOne → User, Workspace; UniqueConstraint(user_id, workspace_id)
Space	id (UUID), name, description, color, isPrivate	ManyToOne → Workspace; OneToMany → Folder
Folder	id (UUID), name, description, isHidden, createdAt	ManyToOne → Space; OneToMany → Liste
Sprint	id (UUID), name, startDate, endDate, goal, isActive	OneToMany → Liste
Liste	id (UUID), name, type (ListType), order, createdAt	ManyToOne → Folder, Sprint; OneToMany → Task
Task	id (UUID), title, description (TEXT), status (TaskStatus), priority (Priority), dueDate, createdAt, updatedAt	ManyToOne → Liste, Sprint, User (assignee);
Conversation	id (UUID), lié à un contexte IA	OneToMany → ConversationMessage
Workspace-Invitation	id, email, token, status (InvitationStatus), expiresAt	ManyToOne → Workspace

3.3.3 Énumérations métier

Enum	Valeurs
TaskStatus	TO_DO, IN_DEV, IN_TEST, IN_REVIEW, DONE
Priority	LOW, MEDIUM, HIGH, URGENT
ListType	SPRINT, PHASE
WorkspaceRole	OWNER, ADMIN, MEMBER
InvitationStatus	PENDING, ACCEPTED, EXPIRED

TABLE 3.2 – Énumérations JPA du modèle de données

3.4 Patterns architecturaux adoptés

La conception du système s’appuie sur plusieurs patterns éprouvés du génie logiciel dont voici les principaux.

Layered Architecture (Backend)

Séparation stricte en couches : **web** (Controllers REST), **service** (Managers et interfaces), **dao** (entités JPA et Repositories Spring Data) et **dto** (objets de transfert avec mapping ModelMapper).

DTO Pattern

Chaque entité expose des **RequestDto** (entrée) et des **ResponseDto** (sortie) distincts, évitant toute fuite de données sensibles (ex : hash de mot de passe) et découplant le modèle de persistance du contrat API.

RAG — Retrieval-Augmented Generation (Service IA)

Le pipeline IA récupère les fichiers pertinents d’un dépôt GitHub, les injecte comme contexte dans le prompt LLM, et génère une réponse structurée. Ce pattern évite les hallucinations et ancre les réponses dans du code réel.

Singleton LLM (Service IA)

L’instance **ChatOpenAI** est mise en cache via **@lru_cache** pour éviter l’initialisation répétée du client LLM à chaque requête.

Token caching (Frontend)

Le module **jwtService.tsx** implémente un mécanisme de *refresh-in-flight* : si plusieurs requêtes échouent simultanément, un seul appel de rafraîchissement est émis, les autres attendant le résultat via la même **Promise**.

Chapitre 4

Implémentation Backend — Spring Boot

4.1 Stack technique

Le backend de la plateforme est développé en **Java 17** avec **Spring Boot 4.0.5**. Ce choix garantit une base robuste, un écosystème mature et une compatibilité éprouvée avec Jakarta EE. Le tableau suivant récapitule les dépendances principales extraites du fichier `pom.xml`.

Bibliothèque	Version	Rôle
Spring Boot Parent	4.0.5	Gestionnaire de dépendances central
Spring Data JPA	(géré)	Persistance ORM via Hibernate
Spring Boot Web MVC	(géré)	Contrôleurs REST et sérialisation JSON
Spring Security	(géré)	Authentification, autorisation, filtre JWT
Spring Boot Mail	(géré)	Envoi d'emails (invitations, OTP)
PostgreSQL JDBC Driver	(géré)	Connecteur base de données relationnelle
jjwt-api / impl / jackson	0.11.5	Génération et validation des tokens JWT
ModelMapper	3.2.2	Mapping entités ↔ DTOs
Lombok	(géré)	Génération de code boilerplate (getters, etc.)
Springdoc OpenAPI	2.8.8	Documentation Swagger UI interactive

TABLE 4.1 – Dépendances principales du backend

4.2 Structure du projet

Le projet suit une architecture en couches bien définie hébergée dans le package racine `com.backend.backend`. La séparation des responsabilités est assurée par les sous-packages suivants.

- `web/` — 13 contrôleurs REST exposant l'API publique.
- `service/manager/` — Managers encapsulant la logique métier.
- `service/serviceInterface/` — Interfaces Java définissant les contrats de service.
- `service/security/` — Service `JwtService` (génération, validation, extraction de claims).

- dao/entities/ — 12 entités JPA persistées en PostgreSQL.
- dao/repositories/ — Repositories Spring Data JPA.
- dao/enums/ — 5 enums métier (TaskStatus, Priority, etc.).
- dto/ — Objets de transfert (Request/Response) par domaine.
- config/ — Configuration Spring Security, CORS, OpenAPI, ModelMapper.
- mapper/ — Mappers spécialisés pour les conversions complexes.

4.3 Sécurité JWT et MFA

4.3.1 Filtre d'authentification JWT

La sécurité de l'API est orchestrée par la classe `JwtAuthenticationFilter`, un filtre Spring héritant de `OncePerRequestFilter`. À chaque requête entrante, il extrait le token du header `Authorization: Bearer <token>`, valide sa signature et son expiration via le `JwtService`, puis peuple le contexte Spring Security avec l'email et le rôle de l'utilisateur.

```
1 @Component
2 @RequiredArgsConstructor
3 public class JwtAuthenticationFilter extends
4     OncePerRequestFilter {
5
6     private final JwtService jwtService;
7
8     @Override
9     protected void doFilterInternal(HttpServletRequest request,
10                                     HttpServletResponse
11                                     response,
12                                     FilterChain filterChain)
13         throws ServletException,
14         IOException {
15         String authHeader = request.getHeader(HttpHeaders.
16             AUTHORIZATION);
17
18         if (authHeader != null && authHeader.startsWith("Bearer
19             ")) {
20             String token = authHeader.substring(7);
```

```
16
17         if (jwtService.isValidAccessToken(token)
18             && SecurityContextHolder.getContext()
19                 .getAuthentication()
20                     == null) {
21             String email = jwtService.extractSubject(token)
22                 ;
23             String role = jwtService.extractRole(token);
24
25             UsernamePasswordAuthenticationToken
26                 authentication =
27                 new UsernamePasswordAuthenticationToken(
28                     email, null,
29                     Collections.singletonList(
30                         new SimpleGrantedAuthority("ROLE_"
31                             + role)));
32             authentication.setDetails(
33                 new WebAuthenticationDetailsSource().
34                     buildDetails(request));
35             SecurityContextHolder.getContext()
36                 .setAuthentication(
37                     authentication);
38         }
39     }
40     filterChain.doFilter(request, response);
41 }
42
43 @Override
44 protected boolean shouldNotFilter(HttpServletRequest
45     request) {
46     // Les routes /api/auth/** sont exclues du filtre JWT
47     return request.getServletPath().startsWith("/api/auth")
48         ;
49 }
50 }
```

Listing 4.1 – Extrait du filtre JWT — JwtAuthenticationFilter.java

4.3.2 Configuration Spring Security

La classe `SecurityConfig` configure la chaîne de filtres en mode `STATELESS` (aucune session HTTP), active le support CORS, désactive la protection CSRF (inutile en contexte JWT) et intègre le `JwtAuthenticationFilter` avant le filtre standard de Spring.

```
1 @Configuration
2 @EnableMethodSecurity
3 @RequiredArgsConstructor
4 public class SecurityConfig {
5
6     @Bean
7     public PasswordEncoder passwordEncoder() {
8         return new BCryptPasswordEncoder();
9     }
10
11     @Bean
12     public SecurityFilterChain securityFilterChain(HttpSecurity
13         http)
14
15                                     throws
16                                     Exception
17                                     {
18
19         http
20             .cors(Customizer.withDefaults())
21             .csrf(csrf -> csrf.disable())
22             .sessionManagement(session ->
23                 session.sessionCreationPolicy(
24                     SessionCreationPolicy.STATELESS))
25             .authorizeHttpRequests(auth -> auth
26                 .requestMatchers(HttpMethod.OPTIONS, "**").
27                     permitAll()
28                 .requestMatchers("/api/auth/**",
29                     "/swagger-ui/**", "/v3/api-docs/**").
30                     permitAll()
31                 .anyRequest().authenticated())
32             .addFilterBefore(jwtAuthenticationFilter,
33                 UsernamePasswordAuthenticationFilter.class);
34 }
```

```

26     return http.build();
27 }
28 }

```

Listing 4.2 – Configuration de la chaîne de sécurité — SecurityConfig.java

Remarque

La MFA est implémentée au niveau applicatif dans la couche service. Lors de la connexion, si `mfaEnabled = true`, le backend génère un OTP haché via BCrypt, le stocke avec une date d'expiration dans `mfaOtpExpiresAt`, et redirige le client vers le challenge MFA (POST `/api/auth/mfa/verify`). Le champ `mfaChallengeId` identifie la session de challenge en cours et `mfaOtpAttempts` limite les tentatives.

4.4 Endpoints API principaux

L'API REST expose **13 contrôleurs** couvrant l'ensemble du domaine métier. Le tableau suivant recense les routes les plus significatives.

Méthode	Route	Description
Authentification — AuthController		
POST	<code>/api/auth/register</code>	Inscription d'un nouvel utilisateur
POST	<code>/api/auth/login</code>	Connexion, retourne Access + Refresh tokens
POST	<code>/api/auth/mfa/verify</code>	Vérification du code OTP MFA
POST	<code>/api/auth/refresh</code>	Rafraîchissement du token d'accès
POST	<code>/api/auth/logout</code>	Révocation du refresh token
Workspaces — WorkspaceController		
GET	<code>/api/workspaces</code>	Workspaces de l'utilisateur courant
POST	<code>/api/workspaces</code>	Création d'un workspace
PUT	<code>/api/workspaces/{id}</code>	Mise à jour d'un workspace
DELETE	<code>/api/workspaces/{id}</code>	Suppression d'un workspace

(suite page suivante)

Méthode	Route	Description
Membres — WorkspaceMemberController		
POST	/api/workspaceMembers/invite	Invitation par email
PUT	/api/workspaceMembers/{id}	Mise à jour du rôle d'un membre
Spaces — SpaceController		
POST	/api/spaces	Création d'un space
GET	/api/spaces/workspace/{id}	Spaces d'un workspace
Sprints — SprintController		
POST	/api/sprints	Création d'un sprint
PUT	/api/sprints/{id}	Mise à jour d'un sprint
Tâches — TaskController		
POST	/api/tasks	Création d'une tâche
GET	/api/tasks/liste/{id}	Tâches d'une liste
GET	/api/tasks/sprint/{id}	Tâches d'un sprint
PUT	/api/tasks/{id}	Mise à jour complète
PATCH	/api/tasks/{id}/assign/{aId}	Assignment d'une tâche
PATCH	/api/tasks/{id}/sprint/{sId}	Déplacement vers un sprint
DELETE	/api/tasks/{id}	Suppression d'une tâche
Listes — ListeController		
POST	/api/listes	Création d'une liste
GET	/api/listes/folder/{id}	Listes d'un dossier
GET	/api/listes/sprint/{id}	Listes d'un sprint
Notifications — NotificationController		
POST	/api/notifications	Création d'une notification
GET	/api/notifications/{id}	Récupération d'une notification par son ID
GET	/api/notifications	Récupération de toutes les notifications

(suite page suivante)

Méthode	Route	Description
GET	/api/notifications/user/{userId}	Notifications d'un utilisateur
GET	/api/notifications/user/{userId}/unread	Notifications non lues d'un utilisateur
PATCH	/api/notifications/{id}/read	Marquer une notification comme lue
PATCH	/api/notifications/user/{userId}/read-all	Marquer toutes les notifications d'un utilisateur comme lues
DELETE	/api/notifications/{id}	Suppression d'une notification
Intelligence Artificielle — FastAPI (PM-B-ia)		
POST	/api/ia/generate	Génération d'entité par IA
POST	/api/ia/repo	Analyse RAG d'un dépôt GitHub
POST	/api/ia/validate	Validation d'un dépôt GitHub

Chapitre 5

Implémentation Frontend — React

5.1 Stack technique frontend

L'interface utilisateur est une **Single Page Application** développée avec l'écosystème React moderne. L'ensemble des versions ci-dessous est extrait du fichier `package.json` du projet.

Bibliothèque	Version	Rôle
React	19.2.4	Framework UI, gestion des composants et du DOM virtuel
React DOM	19.2.4	Rendu dans le navigateur
TypeScript	~6.0.2	Typage statique de l'ensemble du code
Vite	8.0.4	Bundler ultra-rapide avec HMR
TailwindCSS	4.2.2	Framework CSS utility-first
React Router DOM	7.14.0	Routage côté client (SPA)
@dnd-kit/core	6.3.1	Moteur de Drag & Drop accessible
@dnd-kit/sortable	10.0.0	Abstraction de tri pour les listes Kanban
@dnd-kit/utilities	3.2.2	Utilitaires CSS pour transitions dnd-kit
Framer Motion	12.38.0	Animations et transitions fluides
Lucide React	0.474.0	Librairie d'icônes SVG cohérentes
react-markdown	10.1.0	Rendu Markdown (réponses IA, descriptions)
remark-gfm	4.0.1	Extension GitHub Flavored Markdown

TABLE 5.1 – Stack technique frontend (extrait `package.json`)

5.2 Architecture des pages et routage

Le fichier `App.tsx` orchestre le routage de l'application via `react-router-dom v7`. La composante `ProtectedRoute` vérifie la présence d'un `accessToken` en `localStorage` avant d'autoriser l'accès aux pages protégées, et redirige vers `/login` en cas d'absence.

Route	Composant Page	Accès
/login	LoginPage.tsx	Public
/register	RegisterPage.tsx	Public
/invite	InvitationWelcomePage.tsx	Public
/invite/register	InviteRegisterPage.tsx	Public
/github/callback	GitHubCallbackPage.tsx	Public
/workspace	DashboardPage.tsx	Protégé
/ai	AIPage.tsx	Protégé

TABLE 5.2 – Pages de l’application et leur statut d’accès

5.3 Composants clés

L’ensemble des composants réutilisables est organisé dans `src/components/`. La taille des fichiers témoigne de la richesse fonctionnelle de chaque composant.

WorkspaceResourcesPanel.tsx

Panneau central de gestion des ressources d’un workspace. Gère les formulaires de création et de modification des Spaces, Folders, Lists, Sprints et Tasks. Communique les événements de création au parent `DashboardPage` pour déclencher la resynchronisation de l’état.

BoardView.tsx

Tableau Kanban complet utilisant `@dnd-kit/core` et `@dnd-kit/sortable`. Organise les tâches en colonnes correspondant aux statuts (TO_DO, IN_DEV, IN_TEST, IN_REVIEW, DONE). Le déplacement d’une carte déclenche un appel `PUT /api/tasks/{id}` mettant à jour le statut.

Sidebar.tsx

Barre latérale de navigation hiérarchique affichant les Workspaces, Spaces, Folders et Lists. Gère l’état d’expansion/collapse de chaque nœud et le surlignage de la ressource active.

WorkspaceTopBar.tsx

Barre supérieure contextuelle affichant les métadonnées du workspace actif, le menu utilisateur et les actions rapides.

WorkspacesDropdown.tsx

Sélecteur de workspace permettant de basculer entre les différents espaces de

travail de l'utilisateur connecté.

TaskForms.tsx

Formulaires modaux de création et d'édition de tâches avec sélection de priorité, statut, assignation et date d'échéance.

listeForms.tsx

Formulaires de gestion des Listes, permettant la création de colonnes Kanban de type SPRINT ou PHASE.

ListView.tsx

Vue alternative au Kanban : affichage des tâches sous forme de liste tabulaire avec tri et filtres.

ViewNavBar.tsx

Barre de navigation des vues (Board, List) avec indicateur de vue active.

5.4 Implémentation du Kanban — dnd-kit

Le cœur de l'expérience utilisateur repose sur la bibliothèque **@dnd-kit**. Cette librairie a été privilégiée pour son accessibilité native, son support TypeScript exhaustif et sa flexibilité par rapport aux alternatives comme React-Beautiful-DnD (déprécié).

Le composant **BoardView** enveloppe les colonnes dans un contexte **DndContext**. Chaque carte de tâche est rendue dans un composant **SortableContext**, disposant d'un identifiant unique correspondant à l'UUID de la tâche. L'événement **onDragEnd** intercepte la fin du glissement, identifie la colonne cible (nouveau statut), et déclenche un appel HTTP PUT `/api/tasks/{id}` avec le statut mis à jour.

5.5 Gestion des appels API et authentification frontend

Les appels vers le backend sont organisés dans `src/api/` sous forme de modules TypeScript spécialisés : `authApi.tsx`, `taskApi.tsx`, `sprintApi.tsx`, `spaceApi.tsx`, `iaApi.tsx`, etc.

Le module `jwtService.tsx` centralise la gestion des tokens. Il implémente un mécanisme de **token refresh transparent** : avant chaque requête protégée, il vérifie la validité de l'access token via `GET /api/ping`. En cas d'expiration, il effectue un

POST `/api/auth/refresh` automatique. Un verrou `refreshInFlight` (type `Promise`) garantit qu'un seul refresh est émis même si plusieurs requêtes échouent simultanément.

Remarque

La page `AIPage.tsx` est le composant le plus volumineux du projet. Elle intègre l'interface complète de l'assistant IA : gestion des dépôts GitHub, conversations persistantes, panneau de génération d'entités, et rendu des réponses en Markdown via `react-markdown` avec le plugin `remark-gfm` pour le support des tableaux et des blocs de code GitHub.

Chapitre 6

Intelligence Artificielle

6.1 Vision fonctionnelle — Ce que l’IA apporte à l’utilisateur

Avant d’aborder l’implémentation technique, il est essentiel de comprendre la valeur concrète que le module d’intelligence artificielle apporte aux utilisateurs de la plateforme, indépendamment de leur profil technique.

L’assistant IA d’Orbyte remplit **trois grandes missions** du point de vue de l’utilisateur final.

6.1.1 Mission 1 — Comprendre instantanément n’importe quel projet de code

Un chef de projet ou un nouveau développeur rejoignant une équipe doit souvent passer plusieurs jours à explorer le code source, lire la documentation technique et interroger les membres de l’équipe pour appréhender l’architecture générale d’un projet. Avec l’assistant IA, il suffit de **connecter le dépôt GitHub du projet** (qu’il soit public ou privé) et de poser des questions en langage naturel, comme s’il s’adresse à un expert technique du projet.

Exemple de dialogue — Compréhension de projet

Utilisateur : « *Comment est gérée l’authentification dans ce projet ?* »

Assistant Orbyte IA : « L'authentification repose sur des tokens JWT. L'utilisateur se connecte via `/api/auth/login`, reçoit un access token (durée courte) et un refresh token (longue durée). Un filtre Spring intercepte chaque requête pour valider le token. Si l'utilisateur a activé la MFA, un code OTP lui est envoyé par email avant d'accéder à l'application. »

Cette capacité repose sur l'analyse dynamique du code source réel du dépôt GitHub fourni par l'utilisateur. L'assistant ne fait pas d'hallucinations basées sur des connaissances génériques; il lit le code source ciblé et en renvoie une réponse précise et contextualisée.

6.1.2 Mission 2 — Créer des éléments de projet structurés par le texte (Wizard multi-entités)

La planification de sprints, la structuration de dossiers ou la création répétitive de tâches constituent des opérations chronophages pour les équipes agiles. L'assistant IA d'Orbyte permet de **générer des arborescences complètes et des entités applicatives complexes en une seule phrase** en langage naturel.

Grâce à son moteur de génération hiérarchique, l'assistant comprend les relations de parenté et orchestre un flux d'étapes ordonné. L'utilisateur n'est pas confronté à une création opaque : le frontend affiche un **wizard de confirmation interactif (AIConfirmCard)** permettant de passer en revue, modifier et valider chaque entité générée avant son insertion en base de données.

Exemple de dialogue — Génération hiérarchique d'entités

Utilisateur : « Crée un espace de travail "Développement", un dossier "Sprint 1", une liste de tâches, et dedans : configurer Nginx et rédiger la documentation. »

Assistant Orbyte IA : « J'ai préparé un plan de création structuré en 5 étapes pour votre projet. Vous pouvez valider chaque étape ou modifier les détails directement ci-dessous :

1. **Space** : Création de l'espace "Développement"
2. **Folder** : Création du dossier "Sprint 1"
3. **Liste** : Création de la liste des tâches

4. **Task** : Configuration du serveur Nginx

5. **Task** : Rédaction de la documentation technique

»

[Affichage de l'assistant interactif (Wizard) pré-remplissant les formulaires successifs avec résolution dynamique des IDs parents]

L'IA pré-remplit les champs requis, gère les dépendances logiques (les UUIDs des entités parentes créées aux étapes 1, 2 et 3 sont automatiquement injectés dans les entités enfants des étapes suivantes), et transmet les données au backend Spring Boot de façon transparente.

6.1.3 Mission 3 — Générer du contenu ancré dans le contexte réel du code

Lorsqu'un développeur travaille sur un projet dont le dépôt GitHub est connecté, l'assistant peut **croiser la demande de l'utilisateur avec le code source réel** pour générer des descriptions techniques, des sous-tâches ou des scénarios de test pertinents et ancrés dans l'implémentation existante.

Exemple de dialogue — Génération contextuelle

Utilisateur : « En te basant sur le code de notre dépôt, génère les tâches requises pour améliorer la couverture de tests du module de sécurité. »

Assistant Orbyte IA : « Après analyse des fichiers `JwtAuthenticationFilter.java` et `SecurityConfig.java`, je propose les tâches suivantes : (1) Tester le comportement du filtre en cas de token JWT expiré, (2) Valider le rejet des requêtes sans en-tête d'autorisation, (3) Vérifier le traitement des certificats SSL auto-signés. »

6.2 Les fonctionnalités IA — Vue d'ensemble

Le tableau suivant synthétise les fonctionnalités IA de la plateforme Orbyte, intégrant les évolutions de sécurité et de génération hiérarchique.

Fonctionnalité	Bénéfice utilisateur	Exemple concret
----------------	----------------------	-----------------

Analyse de dépôts GitHub		Comprendre l'architecture globale et le code source de dépôts publics ou privés en quelques secondes.	« Explique-moi le fonctionnement de la validation des formulaires. »
Génération hiérarchique		Initialiser des structures complexes et ordonnées (Workspace → Space → Folder → Liste → Task).	« Crée un nouveau projet de test avec un espace de dev et 2 tâches. »
Formulaires dynamiques		Éditer et valider pas à pas via un Wizard de confirmation interactif (AIConfirmCard) évitant les actions opaques.	Validation séquentielle de formulaires pré-remplis avec correction à la volée.
Résolution de dépendances		Gestion des relations de parenté entre entités générées non encore persistées (UUIDs marqués <code>__PENDING__</code>).	L'ID du Space tout juste créé est injecté dans le Folder à l'étape suivante.
Synchronisation Swagger		Extraction dynamique des endpoints et structures du backend via <code>/v3/api-docs</code> pour éviter toute désynchronisation.	Prise en compte immédiate de l'ajout d'un champ ou d'une validation dans l'API Java.
Génération de contenu (Ask AI)		Auto-complétion intelligente des descriptions d'entités ou des objectifs de sprint depuis leur nom via un PO virtuel.	Clic sur l'icône « Ask IA » pour rédiger la description d'une tâche de déploiement.

Validation des branches	Correction automatique ou alerte en cas de branche inexistante (fallback sur la branche par défaut).	Redirection d’une requête sur main vers la branche active réelle master .
Sécurité renforcée des PAT	Chiffrement symétrique Fernet des tokens GitHub côté IA. Stockage en PostgreSQL et transit sécurisé exclusif.	Déchiffrement temporaire du token en mémoire vive (RAM) lors de l’accès à GitHub.

TABLE 6.2 – Fonctionnalités IA et intégration utilisateur

6.3 Architecture du service IA

Le module d’intelligence artificielle est un microservice autonome développé en **Python 3** avec le framework **FastAPI 0.111.0** (version applicative 3.0.0). Son isolation dans un service dédié (**PM-B-ia**) permet de le faire évoluer indépendamment du backend Java Spring Boot, de le scaler séparément selon la charge de requêtes, et de maintenir un environnement propre aux bibliothèques de NLP.

Le serveur est exécuté via le serveur ASGI **Uvicorn 0.29.0**, qui assure une gestion asynchrone performante. La validation et la sérialisation des données d’entrée/sortie sont garanties par **Pydantic 2.7.1**.

Le tableau 6.3 répertorie les dépendances clés du service IA.

Bibliothèque	Version	Rôle
FastAPI	0.111.0	Framework API asynchrone (ASGI)
Uvicorn	0.29.0	Serveur ASGI haute performance
Pydantic	2.7.1	Validation et sérialisation des données
LangChain	0.2.1	Orchestration des requêtes LLM
langchain-openai	0.1.8	Connecteur pour l'API LLM (OpenRouter / OpenAI)
langchain-text-splitters	0.2.0	Outil optionnel de segmentation de texte
requests	2.31.0	Appels HTTP synchrones (API GitHub REST)
httpx	0.27.0	Client HTTP asynchrone (communication inter-services)
python-dotenv	1.0.1	Chargement des variables d'environnement
python-multipart	0.0.9	Gestion des téléversements de données formulaires
cryptography	42.0.8	Chiffrement symétrique Fernet des PAT GitHub

TABLE 6.3 – Dépendances du service IA (extrait `requirements.txt`)

6.3.1 Sécurité des Personal Access Tokens GitHub

L'accès sécurisé aux dépôts privés GitHub s'appuie sur une architecture de chiffrement à double barrière, implémentée dans `services/encryption_service.py` :

1. **Transit sécurisé** : Le frontend transmet le Personal Access Token (PAT) de l'utilisateur sous forme brute uniquement au moment de l'enregistrement ou de la

validation (POST /api/ia/repos/add) via une connexion HTTPS chiffrée.

2. **Chiffrement immédiat** : Dès réception, le service IA chiffre le token en mémoire vive avec l'algorithme **Fernet** (chiffrement symétrique basé sur AES-128-CBC et signature HMAC-SHA256). La clé de chiffrement (`ENCRYPTION_KEY`) est stockée uniquement dans le fichier `.env` du serveur IA. Elle ne transite jamais sur le réseau et n'est pas stockée en base de données.
3. **Persistance déchargée** : Seule la chaîne chiffrée obtenue est transmise au backend Spring Boot via l'endpoint POST /api/repos/upsert. Ainsi, la base de données PostgreSQL de production ne contient **jamais** de tokens d'accès GitHub en clair.
4. **Déchiffrement éphémère (RAM-only)** : Lors d'une requête d'analyse de dépôt, le service IA récupère le token chiffré depuis le backend Java, le déchiffre en mémoire vive uniquement pour réaliser les appels à l'API GitHub, puis détruit la variable en fin de traitement. Le token en clair ne transite jamais entre les microservices.

6.4 Les endpoints REST exposés

Le service expose **six endpoints REST** dans `main.py`, structurés pour séparer la gestion des dépôts et les services cognitifs de l'IA.

Méthode	Endpoint	Rôle
POST	/api/ia/repos/add	Validation d'existence du dépôt et de la branche, chiffrement du PAT et persistance via le backend Java.
DELETE	/api/ia/repos/delete	Suppression d'un dépôt enregistré et de ses jetons associés.
POST	/api/ia/validate	Validation à la volée des jetons et de l'accessibilité d'un dépôt GitHub.
POST	/api/ia/repo	RAG : Analyse de dépôts GitHub et génération d'une réponse technique à la question utilisateur.
POST	/api/ia/generate	Détection d'intentions de création d'entités (séquentielles ou hiérarchiques) et réponse sous forme de flux JSON.
POST	/api/ia/ask-ai	Génération d'un paragraphe descriptif / objectif agile pour une entité spécifique à partir de son nom.

TABLE 6.4 – Endpoints REST du service IA (`main.py`)

Le modèle par défaut configuré via la passerelle d'accès LLM est `openai/gpt-oss-120b:free` via **OpenRouter** (`openrouter.ai/api/v1`), ajustable de manière centralisée dans le fichier `config/config.py`.

6.5 Intégration GitHub — `github_service.py`

Le module `services/github_service.py` gère l'ensemble des interactions avec l'API REST v3 de GitHub. Pour préserver la fluidité de la boucle d'événements asynchrone de FastAPI, toutes les requêtes réseau synchrones de la bibliothèque `requests` sont déléguées à des threads dédiés via `asyncio.to_thread()`.

Il expose trois fonctionnalités majeures :

fetch_file_tree(owner, repo, branch, token)

Récupère l'arborescence complète du dépôt spécifié. La liste des fichiers est automatiquement filtrée selon des extensions de code autorisées définies en configuration (e.g., .java, .py, .ts, .tsx, .md). L'arbre résultant est mis en cache pendant **5 minutes**.

fetch_file_content(file_path, owner, repo, branch, token)

Télécharge le contenu textuel brut d'un fichier source spécifique. Pour optimiser les quotas de l'API GitHub et réduire le temps de réponse, le contenu est mis en cache en mémoire vive pendant **10 minutes**.

get_latest_commit(owner, repo, branch)

Interroge le SHA du dernier commit pour vérifier la fraîcheur du code indexé.

Le cache mémoire est implémenté sous forme de dictionnaires locaux protégés par des verrous de thread (`threading.Lock()`) pour garantir la thread-safety lors des accès concurrents asynchrones. De plus, le service intègre une validation et une correction automatique de branche : si une branche spécifiée s'avère invalide lors de l'appel GitHub (erreur 404), le service recherche la branche par défaut du dépôt (généralement `main` ou `master`) et l'utilise automatiquement comme fallback, évitant ainsi le blocage de l'application.

6.6 Pipeline de génération contextuelle

Le mécanisme permettant à l'IA de répondre à partir du **vrai contenu du code source** d'un dépôt GitHub repose sur un pipeline d'orchestration de LLM à deux étapes (Two-Stage LLM Selection Pipeline), implémenté dans `services/rag_service.py`.

Contrairement aux approches RAG classiques qui découpent arbitrairement le code dans une base vectorielle (avec des risques de désynchronisation et de perte de contexte structurel), ce pipeline utilise l'intelligence du modèle pour effectuer une **sélection de fichiers contextualisée**. Ce traitement est déclenché par l'endpoint `POST /api/ia/repo`.

Le fonctionnement du pipeline est décrit ci-dessous :

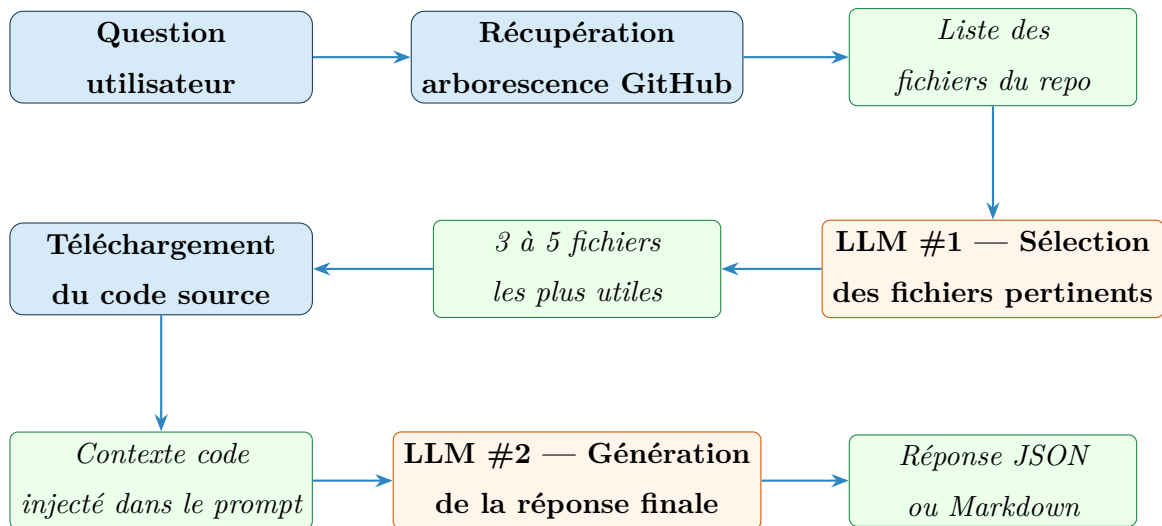


FIGURE 6.1 – Pipeline de génération contextuelle à double LLM (rag_service.py)

Remarque

L'étape critique de ce pipeline est la **sélection intelligente de fichiers** effectuée par le premier LLM (fonction `identify_relevant_files()`). Au lieu d'injecter l'arborescence complète (qui saturerait la fenêtre de contexte) ou de faire une recherche par mots-clés rigide, l'IA détermine par raisonnement sémantique les 3 à 5 fichiers du dépôt les plus à même de contenir la réponse. Cette sélection dynamique permet d'injecter du code brut ciblé et de haute qualité dans le prompt du second LLM, améliorant la pertinence technique tout en limitant les coûts d'inférence.

De plus, le service prend en charge l'analyse **multi-dépôts simultanée**. Les arborescences de plusieurs dépôts connectés sont lues en parallèle via `asyncio.gather()`. Les chemins sont alors préfixés au format `[owner/repo] chemin/fichier` pour que le LLM maintienne le contexte de provenance du code lors de sa synthèse.

6.7 Génération d'entités par intention

La capacité d'Orbyte à traduire une simple phrase en une suite d'entités de gestion de projet (Workspaces, Sprints, Tâches, etc.) repose sur le moteur de génération intelligent de `rag_service.py`. Ce moteur s'appuie sur deux modes de fonctionnement complémentaires.

6.7.1 Mode dynamique — Swagger-driven

Afin d'éviter qu'un changement de modèle de données ou d'API côté backend Spring Boot ne casse le module d'intelligence artificielle, le service IA implémente un mode d'analyse dynamique.

À chaque sollicitation, le service IA effectue une requête HTTP asynchrone sur la route OpenAPI du backend (`GET /v3/api-docs`). Il extrait en temps réel la liste des endpoints de création (`POST`) ainsi que le schéma attendu pour leurs corps de requête (champs obligatoires, types, descriptions). Ce dictionnaire d'API à jour est injecté dans le prompt du LLM. Le modèle sélectionne alors lui-même les routes cibles et construit les structures JSON adaptées, garantissant une synchronisation permanente et sans maintenance du code IA avec les évolutions de l'application Java.

6.7.2 Mode fallback statique

En cas de coupure réseau ou d'indisponibilité temporaire du backend Spring Boot, le service IA bascule automatiquement sur un mode dégradé. Il utilise un dictionnaire de schémas d'entités codé en dur (`ENTITY_SCHEMAS`) et un système d'analyse d'intentions basé sur des correspondances de mots-clés dans la requête de l'utilisateur.

6.7.3 Gestion de la hiérarchie et résolution de dépendances

Lorsqu'un utilisateur demande à créer une structure imbriquée (par exemple, un nouvel espace contenant un dossier, lui-même contenant une tâche), l'assistant doit gérer les dépendances logiques. Les entités parentes n'existant pas encore au moment du prompt, leurs identifiants uniques (UUIDs) sont inconnus.

Le service IA résout ce problème en générant des objets d'étapes dans une clé `flow`. Pour chaque champ requis représentant une clé étrangère non encore créée, le service insère un marqueur temporaire nommé `"__PENDING__"`.

C'est ensuite le frontend, au sein du composant interactif `AIConfirmCard`, qui orchestre la création séquentielle :

1. Le frontend affiche le formulaire de l'entité parente (ex : le Space) pré-rempli pour validation.
2. Dès que l'utilisateur clique sur « Confirmer », le frontend effectue l'appel API réel de création au backend et récupère l'UUID généré par la base de données.

3. Le frontend enregistre cet ID et l'utilise pour remplacer à la volée le marqueur "__PENDING__" du champ parent de l'étape suivante (ex : le champ `spaceId` du Folder) avant de l'afficher à l'utilisateur.

Ce que dit l'utilisateur	Entité créée	Endpoint appelé
« Crée une tâche... »	Task	POST /api/tasks
« Nouveau workspace... »	Workspace	POST /api/workspaces
« Ajoute un espace... »	Space	POST /api/spaces
« Planifie un sprint... »	Sprint	POST /api/sprints
« Crée un dossier... »	Folder	POST /api/folders
« Nouvelle liste / colonne »	Liste	POST /api/listes

TABLE 6.5 – Intentions de création et endpoints correspondants

De plus, le contexte de session courant (ID du workspace actif, membres inscrits pour l'affectation automatique des tâches) est injecté dans le prompt de génération de l'entité. Par exemple, si l'utilisateur dit « *Assigne cette tâche à Ilyass* », le service IA recherche le prénom dans la liste des membres actifs du workspace et injecte directement son UUID dans le champ `assigneeId` du JSON de la tâche.

6.7.4 Génération de contenu agile — Ask AI

Complétant le moteur d'intentions, l'endpoint `POST /api/ia/ask-ai` propose une aide à la rédaction de contenu projet. À partir du type d'une entité et de son nom saisi par l'utilisateur, un LLM configuré avec des directives de Product Owner rédige une description professionnelle synthétique (pour un espace, un dossier ou une tâche) ou un objectif de sprint de valeur (Sprint Goal), directement intégrable par simple clic dans l'application.

Chapitre 7

DevOps & Déploiement

7.1 Vue d'ensemble de l'infrastructure

L'infrastructure PM-B est conçue pour déployer la plateforme Orbyte en environnement de production. Elle repose entièrement sur une approche **Infrastructure as Code (IaC)**, garantissant des déploiements reproductibles, versionnables et automatisés, sans aucune intervention manuelle sur les serveurs.

Remarque

L'ensemble du code d'infrastructure est versionné dans le dépôt **PM-B-infra**, séparé des dépôts applicatifs. Cette séparation garantit que la couche infrastructure évolue indépendamment du code métier et peut être audité distinctement.

7.2 Architecture générale

7.2.1 Topologie réseau

L'infrastructure repose sur **cinq nœuds distincts** hébergés sur **Hetzner Cloud** dans la zone **Falkenstein (fsn1)**, reliés par un réseau privé interne. Le serveur de base de données n'est accessible que depuis les serveurs applicatifs via ce réseau privé, constituant ainsi la première couche d'isolation de sécurité.

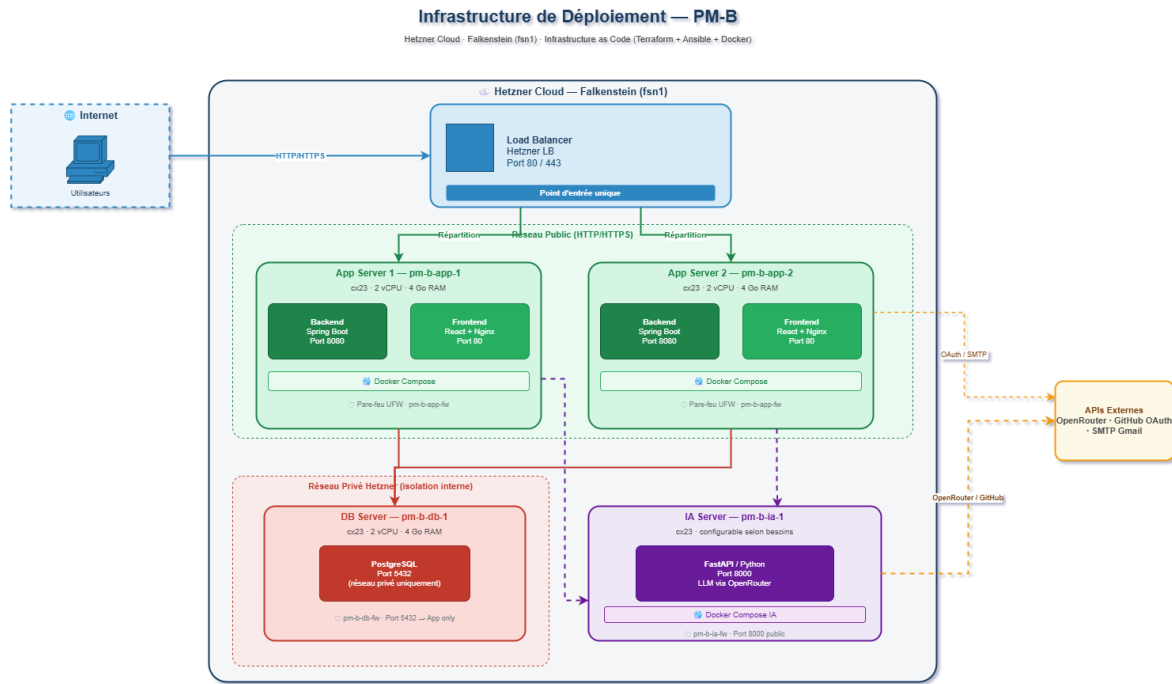


FIGURE 7.1 – Architecture de la topologie réseau — Hetzner Cloud

Composant	Type serveur	Rôle
Load Balancer	Hetzner LB	Point d'entrée unique — distribution du trafic HTTP/HTTPS
App Server ×2	cx23 (2 vCPU / 4 Go)	Backend Spring Boot + Frontend React (Nginx)
DB Server	cx23 (2 vCPU / 4 Go)	Base de données PostgreSQL isolée
IA Server	cx23 (configurable)	Service IA FastAPI / Python

TABLE 7.1 – Topologie des nœuds Hetzner Cloud — PM-B-infra

7.2.2 Structure du dépôt d’infrastructure

Le dépôt PM-B-infra est organisé en trois couches correspondant aux trois outils de la chaîne de déploiement :

```
1 PM-B-infra/  
2 |-- terraform/          # Provisionnement cloud (Hetzner)  
3 |   |-- main.tf         # Definition des ressources  
4 |   |-- variables.tf    # Parametres d'entree  
5 |   |-- outputs.tf      # Adresses IP exportees vers Ansible  
6 |  
7 |-- ansible/           # Configuration des serveurs  
8 |   |-- site.yml        # Playbook principal  
9 |   |-- inventory.ini   # Liste des hotes (remplie apres  
   Terraform)  
10 |   |-- roles/         # Roles modulaires (common, docker, app  
   , db, ia)  
11 |  
12 |-- compose/          # Deploiement applicatif  
13 |   |-- docker-compose.yml    # Backend + Frontend  
14 |   |-- docker-compose.ia.yml # Service IA
```

Listing 7.1 – Arborescence du dépôt PM-B-infra

7.3 Stratégie de conteneurisation

Chaque composant de la plateforme possède son propre **Dockerfile**, garantissant l’isolation totale des environnements d’exécution et la reproductibilité des builds.

Backend Spring Boot

Le **Dockerfile** du backend utilise une image **JDK 17** comme base. Il copie le **JAR** généré par Maven (`mvnw package`) et le lance via `java -jar`. Les variables de configuration (URL base de données, clé **JWT**, **SMTP**) sont injectées via des variables d’environnement au runtime.

Frontend React

Le **Dockerfile** du frontend adopte un pattern **multi-stage build** : une première étape `node:alpine` compile l’application avec `npm run build` (**Vite**), puis une seconde étape copie les assets statiques dans une image **Nginx** légère.

Le fichier `nginx.conf` configure le routage SPA (toutes les routes redirigent vers `index.html`).

Service IA (FastAPI)

L'image Python installe les dépendances depuis `requirements.txt` et démarre Uvicorn sur le port 8000. Les clés API (OpenRouter, GitHub Token) sont injectées via `.env`.

7.4 Infrastructure as Code — Terraform & Ansible

Le déploiement s'articule en **cinq phases** séquentielles et automatisées, chaque phase s'appuyant sur la précédente pour former un pipeline de mise en production cohérent.

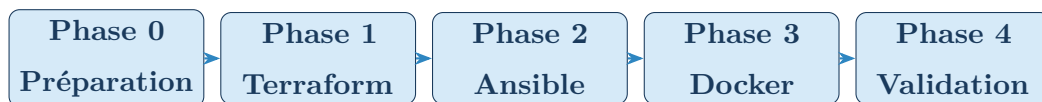


FIGURE 7.2 – Pipeline de déploiement en cinq phases — PM-B-infra

7.4.1 Phase 0 — Préparation locale

Avant tout déploiement, l'environnement de travail local est préparé : installation de Terraform, Ansible et OpenSSH ; génération d'une paire de clés SSH dédiée au projet `pm-b-infra` ; configuration des fichiers `.env` contenant tous les secrets (non versionnés) ; vérification de l'accès à l'API Hetzner Cloud via le token d'authentification.

7.4.2 Phase 1 — Provisionnement Cloud avec Terraform

Terraform crée l'ensemble des ressources cloud à partir de code déclaratif, en utilisant le provider Hetzner Cloud v1.48+ :

- **Clé SSH** enregistrée dans Hetzner pour l'accès aux serveurs.
- **Trois pare-feux** distincts (App, DB, IA) avec règles de filtrage spécifiques.
- **Quatre serveurs** : 2 App + 1 DB + 1 IA, tous en type `cx23`.
- **Export des IPs** (publiques et privées) vers Ansible via `outputs.tf`.

Pare-feu	Port	Accès autorisé
pm-b-app-fw	22 / 80	SSH + HTTP public
pm-b-db-fw	22 / 5432	SSH + PostgreSQL depuis serveurs App uniquement
pm-b-ia-fw	22 / 8000	SSH + API Python publique

TABLE 7.2 – Règles de pare-feu par composant

7.4.3 Phase 2 — Configuration des serveurs avec Ansible

Une fois les serveurs créés, Ansible les configure automatiquement en appliquant des **rôles modulaires** ciblés par groupe de serveurs via le playbook `site.yml` :

- **common** — Base identique pour tous les serveurs : utilisateurs, SSH durci, pare-feu UFW.
- **docker** — Installation de Docker et Docker Compose sur les serveurs App et IA.
- **app** — Dépôt du `docker-compose.yml` et des variables d’environnement.
- **db** — Installation et sécurisation de PostgreSQL, création des utilisateurs et des bases.
- **ia** — Préparation de l’environnement Python / Docker pour le service IA.

L’inventaire Ansible (`inventory.ini`) est peuplé dynamiquement avec les adresses IP exportées par Terraform à l’étape précédente, assurant une continuité sans copier-coller manuel.

7.4.4 Phase 3 — Déploiement applicatif avec Docker Compose

Les applications sont conteneurisées et déployées via deux fichiers Compose distincts selon le serveur cible :

docker-compose.yml (serveurs App)

Lance le backend Spring Boot et le frontend React servi par Nginx. Les deux services communiquent sur un réseau Docker interne. La configuration sensible est injectée via `backend.env` (JWT, base de données, SMTP, OAuth GitHub).

docker-compose.ia.yml (serveur IA)

Lance le service FastAPI exposé sur le port 8000, avec accès au backend

via la variable `BACKEND_BASE_URL`. Les secrets sont injectés via `ia.env` (clés OpenRouter, clé d'encryption, URL backend).

7.5 Sécurité de l'infrastructure

La sécurité repose sur plusieurs niveaux complémentaires et indépendants.

7.5.1 Isolation réseau

Les serveurs communiquent entre eux via le **réseau privé Hetzner**, invisible depuis l'internet public. Le serveur de base de données ne possède aucune interface publique directe : il est uniquement joignable par les serveurs applicatifs sur le port 5432 via ce réseau privé.

7.5.2 Gestion des secrets

Aucun secret n'est versionné dans le dépôt Git. Les fichiers `.env` sont listés dans `.gitignore` et des templates `.env.example` servent de référence documentaire. Les secrets couvrent : clés JWT, mots de passe PostgreSQL, tokens GitHub OAuth et OpenRouter, clés d'encryption du service IA.

Les tokens et credentials ne sont **jamais** embarqués dans les images Docker. Ils sont injectés exclusivement à l'exécution via des fichiers `.env` déposés par Ansible sur les serveurs cibles. Cette pratique garantit que les images peuvent être publiées sur un registre public sans risque de fuite de données sensibles.

7.5.3 Configuration HTTPS/SSL et domaine

La gestion du domaine `orbytee.tech` est entièrement déléguée aux serveurs DNS de Hetzner. La sécurisation des flux est assurée par un certificat SSL managé (Let's Encrypt), directement configuré sur le Load Balancer, qui gère le cycle de vie du certificat (génération et renouvellement automatique) et applique une redirection stricte du trafic HTTP (port 80) vers HTTPS (port 443).

7.6 Pipeline CI/CD — GitHub Actions

Le cycle de vie logiciel est entièrement automatisé via **GitHub Actions**. À chaque push sur la branche principale, le workflow CI/CD se déclenche et orchestre les étapes suivantes.

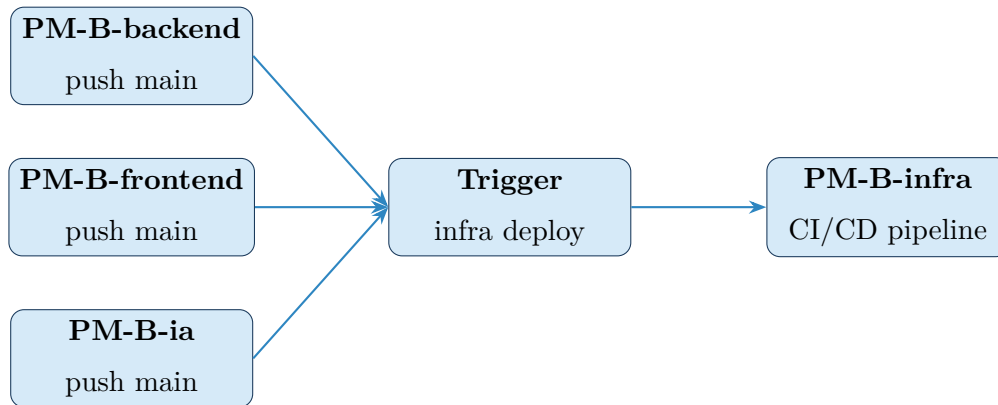


FIGURE 7.3 – Pipeline CI/CD — GitHub Actions vers Hetzner Cloud

Remarque

Afin d’assurer une automatisation complète du déploiement, chaque dépôt applicatif (PM-B-backend, PM-B-frontend, PM-B-ia) dispose d’un workflow GitHub Actions qui se déclenche à chaque push sur la branche main. Ce workflow n’effectue pas directement le déploiement, mais envoie une requête à l’API GitHub pour déclencher le pipeline CI/CD du dépôt central PM-B-infra. Ainsi, toute modification sur l’un des projets entraîne automatiquement le redéploiement de l’infrastructure et des applications, garantissant la cohérence, la traçabilité et la rapidité des mises en production sur l’ensemble de la plateforme.

7.7 Procédures opérationnelles

7.7.1 Déploiement initial

La séquence complète de mise en production s’exécute en deux commandes :

```
1 # 1. Provisionnement de l'infrastructure cloud
2 terraform init && terraform plan && terraform apply
3
4 # 2. Configuration automatique de tous les serveurs
5 ansible-playbook ansible/site.yml -i ansible/inventory.ini
```

Listing 7.2 – Commandes de déploiement complet

7.7.2 Mise à jour applicative

Pour déployer une nouvelle version d'un service, la procédure sur chaque serveur concerné consiste à mettre à jour les images Docker (rebuild ou pull depuis le registre), puis à effectuer un redémarrage à chaud via `docker compose down` suivi de `docker compose up -d`. Cette opération ne nécessite pas de re-provisionner l'infrastructure.

7.7.3 Destruction de l'infrastructure

L'ensemble des ressources Hetzner peut être supprimé en une seule commande `terraform destroy`, facilitant ainsi la gestion des environnements éphémères (staging, tests de charge).

7.8 Évolutions prévues

Évolution	Description	Priorité
Load Balancer actif	Activation complète de la répartition de charge Hetzner entre les deux serveurs App	Moyenne
Auto-scaling	Ajustement dynamique du nombre de serveurs App selon la charge	Moyenne
Monitoring	Déploiement de Prometheus + Grafana pour métriques et alertes	Moyenne
Logs centralisés	Stack ELK pour l'agrégation et l'indexation des logs applicatifs	Basse

TABLE 7.3 – Évolutions prévues de l'infrastructure

Chapitre 8

Tests et validation

8.1 Stratégie globale de tests

La validation de la plateforme Orbyte a été conduite sur l'ensemble de la chaîne technique, du backend jusqu'à l'interface utilisateur, en adoptant une approche multi-niveaux visant à détecter les anomalies au plus tôt dans le cycle de développement.

8.2 Tests backend

8.2.1 Tests d'intégration via requêtes HTTP

Le dossier `src/main/resources/` contient **8 fichiers de tests HTTP** couvrant l'ensemble des domaines métier. Ces scénarios jouables depuis IntelliJ IDEA (client HTTP intégré) ou via des outils comme HTTPie permettent de valider le comportement réel des endpoints.

Fichier de test	Domaine	Scénarios couverts
<code>auth_tests.http</code>	Authentification	Register, login, MFA, refresh, logout
<code>workspace_tests.http</code>	Workspace	CRUD workspace, liste des workspaces
<code>WorkspaceMember_tests.http</code>	Membres	Invitation, rôles, suppression
<code>space_tests.http</code>	Space	Création, mise à jour, suppression
<code>Folder_test.http</code>	Folder	CRUD dossier
<code>liste_tests.http</code>	Liste	CRUD liste, filtrage par folder/sprint
<code>sprint_tests.http</code>	Sprint	Création, modification, pagination
<code>task_tests.http</code>	Task	CRUD tâche, assignation, déplacement sprint

TABLE 8.1 – Fichiers de tests HTTP backend

8.2.2 Validation de la logique de sécurité

Des scénarios dédiés testent les règles d'accès :

- Tentative d'accès sans token JWT → réponse 401 `Unauthorized`.
- Tentative d'accès avec token expiré → rejet par le `JwtAuthenticationFilter`.
- Invitation d'un membre déjà présent → contrainte d'unicité `uk_user_workspace` levée correctement.
- Vérification MFA avec code expiré ou dépassement du nombre de tentatives.

8.3 Tests frontend

8.3.1 Vérification statique TypeScript

La première ligne de défense qualité du frontend est la vérification de types TypeScript. La commande `tsc -noEmit` est exécutée systématiquement lors du pipeline CI/CD. Les fichiers `tsc_errors.txt`, `tsc_final.txt`, `tsc_sync.txt` et `tsc_very_final.txt` documentent le processus itératif de correction des erreurs de typage, notamment celles liées aux types `Partial<TaskRequestDto>` et aux propriétés optionnelles des DTOs.

8.3.2 Analyse statique ESLint

La configuration `eslint.config.js` active les règles `eslint-plugin-react-hooks` et `eslint-plugin-react-refresh`, garantissant le respect des règles d'utilisation des

hooks React et la compatibilité avec le rechargement à chaud Vite.

8.3.3 Tests fonctionnels manuels

Des scénarios de validation bout-en-bout ont été conduits manuellement sur les flux utilisateur les plus critiques :

- Création d'un workspace → vérification de l'apparition immédiate dans la sidebar.
- Invitation d'un membre → réception de l'email, acceptation via `InvitationWelcomePage`, connexion et visibilité du workspace partagé.
- Création d'une tâche via l'assistant IA → détection d'intention, génération JSON, confirmation et envoi au backend.
- Déplacement d'une tâche par Drag & Drop sur le Kanban → mise à jour du statut reflétée immédiatement sans rechargement de page.
- Expiration du token JWT → refresh automatique transparent via `jwtService.tsx`.

8.4 Tableau récapitulatif des tests

Type	Outil	Périmètre	Couverture	Résultat
Intégration API	Fichiers <code>.http</code> (Intel- liJ)	8 domaines, tous les end- points CRUD + sécurité	Complète	OK
Typage sta- tique	<code>tsc -noEmit</code> (TypeS- cript 6)	Tous les fichiers <code>.tsx</code> / <code>.ts</code> du frontend	Complète	OK
Lint React	ESLint react-hooks	+ Composants, hooks, refresh patterns	Complète	OK

Type	Outil	Périmètre	Couverture	Résultat
Test IA RAG	Manuel Postman	+ Endpoint /api/ia/repo et /api/ia/generate	Partielle	OK
Test MFA	Manuel (email OTP)	Flux login com- plet avec et sans MFA	Partielle	OK
Test E2E Kan- ban	Manuel (navi- gateur)	Drag & Drop, mise à jour sta- tut, persistance	Partielle	OK
Test invita- tion	Manuel (email)	Lien, enregistre- ment invité, visi- bilité workspace	Complète	OK
Build Docker	Pipeline CI/CD	3 images (ba- ckend, frontend, ia)	Complète	OK

Conclusion générale

Le projet **Orbyte** a constitué une opportunité rare et exigeante d'expérimenter l'intégralité du cycle de développement d'un système logiciel industriel, de la définition du besoin jusqu'au déploiement automatisé sur infrastructure cloud. En adoptant dès le départ une culture DevOps rigoureuse et en intégrant l'intelligence artificielle comme fonctionnalité de premier rang plutôt que comme ajout tardif, l'équipe a réalisé un produit qui dépasse le cadre académique pour s'inscrire dans les tendances actuelles du marché du logiciel.

Bilan technique

Sur le plan de l'architecture, le découpage en trois microservices indépendants — **PM-B-backend** (Spring Boot), **PM-B-frontend** (React 19 + TypeScript) et **PM-B-ia** (FastAPI + LangChain) — s'est avéré particulièrement judicieux. Il a permis à chaque membre de l'équipe de travailler en parallèle sur son périmètre technique sans créer de blocages, tout en maintenant des interfaces d'intégration bien définies grâce à la documentation OpenAPI générée dynamiquement par Springdoc.

L'innovation technique majeure réside dans le mécanisme de **génération d'entités pilotée par la documentation Swagger** : le service IA interroge en temps réel les spécifications OpenAPI du backend pour choisir le bon endpoint et générer un JSON structuré, garantissant que les suggestions de l'assistant restent toujours synchronisées avec l'implémentation réelle de l'API. Cette approche élimine le risque de divergence entre l'IA et le backend.

Compétences acquises

Ce projet a permis à l'équipe d'approfondir des compétences pluridisciplinaires couvrant :

- La **conception de systèmes distribués** et la gestion des problèmes inhérents

- à ce type d'architecture (synchronisation d'état, CORS, refresh token).
- L'**industrialisation logicielle** via Docker, Terraform, Ansible et les pipelines CI/CD GitHub Actions.
- L'**intégration de LLMs en milieu professionnel** : prompt engineering, patterns RAG, gestion des hallucinations et fallbacks statiques.
- La **sécurité applicative** : JWT, BCrypt, MFA par OTP, gestion des expiration et tentatives ratées.

Perspectives

Trois axes d'évolution majeurs ont été identifiés pour les prochaines versions de la plateforme.

Axe 1 — Collaboration temps réel (WebSocket)

L'implémentation de WebSockets via Spring Boot permettrait d'afficher en temps réel les modifications apportées par d'autres membres de l'équipe sur le tableau Kanban, éliminant le besoin de rafraîchissement manuel et rapprochant l'expérience utilisateur d'outils comme Linear ou Notion.

Axe 2 — Analytique prédictive (Burn-down Charts)

En exploitant l'historique des sprints persistés en base (dates de début, de fin, nombre de tâches, statuts), il serait possible de générer des graphiques de vélocité et des prédictions de complétion par sprint, offrant aux chefs de projet des indicateurs décisionnels objectifs.

Axe 3 — Agents IA autonomes

L'évolution naturelle du module IA consiste à passer d'un assistant conversationnel à des agents capables d'agir : créer automatiquement des tâches à partir d'un commit GitHub, détecter les risques de retard par analyse des patterns de commits, ou proposer une réaffectation de la charge de travail en cas de déséquilibre entre les membres.

En définitive, Orbyte démontre qu'il est possible de concevoir un outil de gestion de projets à la fois léger, sécurisé et augmenté par l'intelligence artificielle, en s'appuyant sur des technologies open-source modernes et des pratiques d'ingénierie logicielle rigoureuses. Ce travail constitue une base solide, immédiatement extensible, pour répondre aux défis de la gestion de projets techniques dans un monde où l'IA devient un partenaire incontournable des équipes de développement.

Annexe A

Diagramme de classes UML

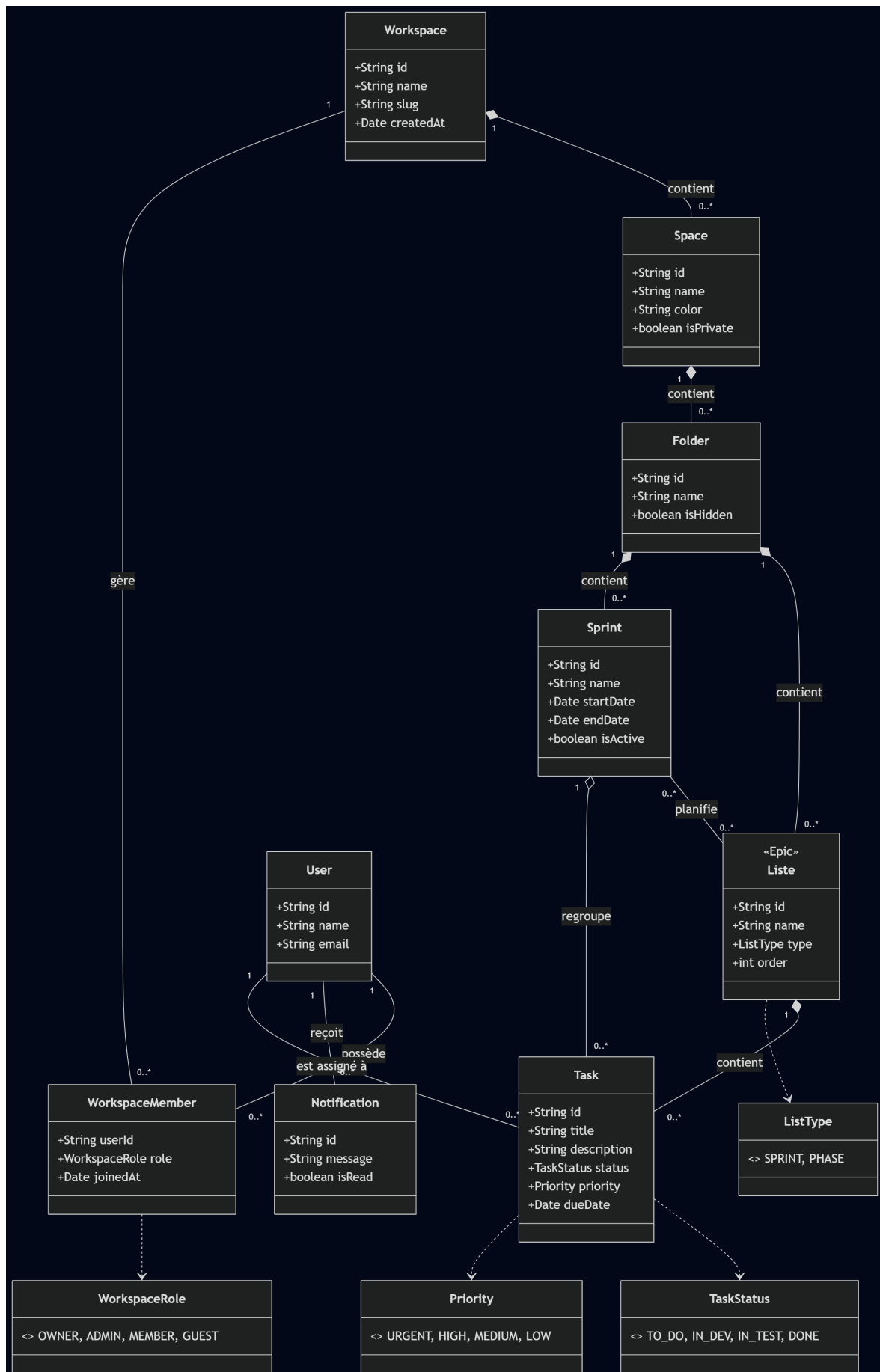


FIGURE A.1 – Diagramme de classes UML complet — Orbyte

Annexe B

Entités JPA — Champs complets

Cette annexe détaille les champs complets de l'entité `Task`, entité centrale du modèle de données.

```
1 @Data @NoArgsConstructor @AllArgsConstructor
2 @Entity @Table(name = "task")
3 public class Task {
4     @Id @GeneratedValue @UuidGenerator
5     @Column(nullable = false, updatable = false)
6     private String id;
7
8     @Column(nullable = false)
9     private String title;
10
11     @Column(columnDefinition = "TEXT")
12     private String description;
13
14     @Enumerated(EnumType.STRING)
15     @Column(nullable = false)
16     private TaskStatus status;    // TO_DO / IN_DEV / IN_TEST /
17                                   IN_REVIEW / DONE
18
19     @Enumerated(EnumType.STRING)
20     @Column(nullable = false)
21     private Priority priority;    // LOW / MEDIUM / HIGH /
22                                   URGENT
23
24     private LocalDateTime dueDate;
```

```
23     private LocalDateTime createdAt;
24     private LocalDateTime updatedAt;
25
26     @ManyToOne(fetch = FetchType.LAZY)
27     @JoinColumn(name = "liste_id", nullable = false)
28     private Liste liste;
29
30     @ManyToOne(fetch = FetchType.LAZY)
31     @JoinColumn(name = "sprint_id")
32     private Sprint sprint;
33
34     @ManyToOne(fetch = FetchType.LAZY)
35     @JoinColumn(name = "assignee_id")
36     private User assignee;
37
38     @OneToMany(mappedBy = "task", cascade = CascadeType.ALL)
39     private List<Subtask> subtasks;
40
41     @OneToMany(mappedBy = "task", cascade = CascadeType.ALL)
42     private List<Comment> comments;
43
44     @OneToMany(mappedBy = "task", cascade = CascadeType.ALL)
45     private List<Attachment> attachments;
46
47     @ManyToMany
48     @JoinTable(name = "task_tag",
49         joinColumns = @JoinColumn(name = "task_id"),
50         inverseJoinColumns = @JoinColumn(name = "tag_id"))
51     private List<Tag> tags;
52 }
```

Listing B.1 – Entité Task.java — champs et relations complètes

Annexe C

Cahier des charges (extrait)

Voir document joint : cahier-de-charges.pdf